

类型	动作	坑点
pointer/array/string	画三格: pointer variable / pointer object / value; 确认 capacity and length.	sizeof array != sizeof ptr; NULL != '\0'; string 必须 NUL.
malloc/realloc	用 sizeof *p; realloc 用 tmp; 所有 valid path 释放已拥有资源.	realloc 失效时旧指针仍有效; heap block 只能 free once; 不要返回局部变量地址; 不要 free 后再读 old->next.
linked list	empty/head/tail/middle; 删除时用 pointer-to-link; free 前保存 next.	feof 不是循环条件; fgets 保留 newline.
file/parsing	优先 fgets+strtol/sscanf; 若用 fscanf, 循环条件必须等于完整字符串数.	mmap 失败是 MAP_FAILED 不是 NULL; mapped bytes 不是 string.
binary/mmap	fread 读完整记录; mmap 用 length 读 bytes.	mmap 失败是 MAP_FAILED 不是 NULL; mapped bytes 不是 string.
fork/exec/wait	child 中 exec; exec 成功不返回; 失败 _exit; parent waitpid.	fork returns twice; child 不要 用 exec 删父进程堆栈; fork EOP 不来或死锁.
pipe/dup2/IPC	一条 pipe 单向; p[0] read, p[1] write; 双向要两条 pipes.	所有不用的 fd 都 close; 否则 EOF 不来或死锁.
thread/mutex/-cond	thread args 用数组/heap; mutex 保护 shared invariant; cond wait 用 while.	cond wait 会 unblock+sleep+relock; 先 predicate 再 signal.
sem/barrier	sem 是计数资源/事件; barrier 是固定人数同步点.	barrier count 错会挂; sem 不是 mutex owner.
security/debug	检查 overflow, format string, UAF, double free, race, command injection.	printf(user), strcpy, system(user) 都危险.

Memory snapshot model: 变量位置、指向位置、值必须分开		
	storage	meaning/trap
local vars / params	stack frame	函数返回后 lifetime 结束; 不能 return pointer to local.
static/global object	static/global segment	whole-program lifetime; static local 作用域局部但值持续.
p = malloc/calloc/realloc;	heap block returned; pointer variable stays where declared	指针变量本身不是 heap; free heap block once.
char *p = "abc";	p in stack/global; literal in read-only data	不能通过 p 修改 literal.
char a[] = "abc";	array contents in array object 位置	local array copy 通常在 stack, 可修.
char x = p[i];	x is char object	复制字符值; x 不是 pointer.
T **pp = malloc(sizeof(T*)); *pp=&obj; **pp rule	heap 存一个 T*; *pp 是 obj 地址	**pp 才是 object value.
	variable location ≠ pointed-to location ≠ pointed-to value	先画 pointer box, 再画 pointer box.

Model Protocol	考试动作	核心坑
	画 memory/process/fd/thread 图 -> owners -> edge cases -> pseudo -> code -> cleanup.	空/堆 /head/tail/middle/error 都测.
argc/argv	argv[0] program; argv[argc]=NULL; 先检查 argc 再索引.	数字用 strtol; 不要信输入.
Pointers	x=value, &x=address, *p=stored address, **p=object, **pp=two-hop object.	地址和值分开; NULL 不是 NUL byte.
Array/ptr	array owns contiguous storage; 参数 T a[] 退化 为 T*.	传 length; sizeof 陷阱.
Struct/ptr array	struct T a[N] 连续对象; struct T *a[N] 连续指针.	对象数组和指针数组不是一回事.
Pointer graph	arr[i] 与 curr->next 是两张图.	array order ≠ linked order.
String	bytes ending '\0'; capacity ≠ length; strlen 不含 NUL.	strncpy may not terminate; ≠ 地址.
Ownership	malloc->free; fopen->fclose; open/pipe->close; popen->pclose; mmap->munmap; pthread_create->join.	每个 acquire 都要 success/fail cleanup.
FILE* vs fd	FILE* buffered stream; fd 是 int OS descriptor; fdopen/fileno 桥梁.	不要随意混 buffered/raw I/O.
Proc/thread	process 地址空间分离; thread 共享 globals/heap; 但各自 stack/registers/IP.	fork duplicates; exec replaces.
Mutex/CS	mutex 保护 shared invariant; 不是保护某一行.	所有访问 shared state 的语句都要同一把锁.
Condvar	condition is shared predicate; wait 原子 unblock/sleep/relock.	必须 while; 先改 predicate 再 signal.
Deadlock	Coffman: mutual exclusion + hold-wait + no preemption + circular wait.	用全局 lock order 破 circular wait.

Struct / Union layout

Item	Rule
alignment	char, uint8=1; uint16=2; uint32/int/float=4; pointer/double usually 8.
field offset	每个 field 从满足自身 alignment 的下一个 offset 开始; 中间可能 padding.
struct size	alignment = max member alignment; sizeof(struct) 向上补齐到 struct alignment 倍数.
union	所有 member 从 offset 0 开始; size = max member size rounded up.
array	arr[i] offset = 1 * sizeof(element); 元素连续.
Security / unsafe C	
Item	Rule
overflow	避免 gets/strcpy/scanf("%s"); 用 fgets, width, size checks.
format heap	printf(user) 危险; 用 printf("%s", user); UAF = free 后访问; double free corrupts heap; realloc 检查 tmp.
command	system/popen + user input 有 command injection; 优先固定 argv 的 execv/execvp.
shared obj	.a static copied into binary; .so runtime loaded; 可能被 path/LD LIBRARY PATH 劫持、替换、ABI mismatch.
race	check 后 用 前状态改变; shared state 用 locks.

Short API: input/string/memory	return / behavior	trap
form	returns assigned count or EOF	参数要地址; %s 加宽逸
int scanf(const char *fmt,...)	从字符串解析; returns assigned count.	混合 fgets 后 parse
int sscanf(const char *str,const char *fmt,...)	最多 n-1 chars + NUL; NULL on EOF/error	保留 \n; n 包含 NUL
char *fgets(char *s,int n,FILE *fp)	char as int or EOF	必须用 int 接收
int getchar(void)	size_t strlen(const char *s)	count before NUL
size_t strlen(const char *s)	int strncmp(strncmp,...)	0 equal; </>0 order
char *strcpy(strcpy,...)	copy string	≠ 地址
void *memcpy(dst,src,n)	copy raw bytes	区域不能 overlap
void *memset(p,c,n)	fill n bytes with byte c	c is byte value
int memcmp(p,q,n)	raw byte compare; 0 equal	不是 string-aware
char *strtok(str,delim)	modifies input; later NULL	hidden state; no thread-safe
void *malloc/calloc(...)	ptr or NULL	用 sizeof *p; calloc zeroes
void *realloc(p,n)	new ptr or NULL	fail 时 old still valid
void free(void *p)	free heap; NULL ok	只能 free once
long strtol(s,&end,base)	convert; end -> first junk	check end + errno
void qsort(base,n,size,cmp)	sort in-place	ERRANGE cmp args point to elements

Short API: file/fd/readiness	return / behavior	trap
form	FILE* or NULL	"/" truncates
FILE *fopen(path,mode)	0 or EOF	之后 invalid
int fclose(FILE *fp)	assigned count or EOF	loop on exact count
int fscanf(FILE *fp,fmt,...)	complete items read	short read: feof/ferror
size_t fread(ptr,size,n,fp)	inspect flags after failed read	不要当循环条件
int feof/ferror(FILE *fp)	flush output	input flush nonportable
int fflush(FILE *fp)	fd >=0 or -1	O' CREAT 需要 mode
int open(path,flags,...)	>0 bytes, 0 EOF; -1 error	partial/EINTR
ssize_t read(fd,buf,n)	bytes written or -1	bytes read
int write(fd,buf,n)	0 or -1	关闭后 invalid
int close(fd)	sets mutate; rebuild	select 前后分别用
int select(rfds,&r,&w,&e,&t)	count, 0 timeout, -1 error	看 revents 不是 events
FD_ZERO/SET/CLR/ISSET	clear/add/remove/test fd	注册 interest; 数组容量
poll(rfds,n,timeout)	count, 0 timeout, -1 error	
epoll_create/createl; cctl; wait	Linux scalable readiness	

Concurrency concepts	Rule
Item	atomic
atomic	不可被中断的完整操作; x+=1 通常是 read+add+write, 非 atomic.
critical section	访问/修改 shared state 的代码区域; 用 mutex 保护.
race	多个线程未同步读写 shared state, 结果依赖执行交错.
deadlock	线程互相等待资源, 全部无法继续; 用固定 lock order 避免.
livelock	线程一直重试/让步但无进展; active but no progress.
starvation	某线程长期得不到锁/CPU/资源; 系统仍运行但它不进展.
contention	很多线程抢同一资源; 导致 Blocking、context switch、并行度下降.

Short API: process/IPC/mmap

form	return / behavior	trap
pid_t fork(void)	child 0; parent child pid; fail -1	returns twice
execv/execvp(path/file,argv)	success no return; fail -1	argv NULL-terminated
execcl/execlp(path/file,argv,...)	success no return	最后必须 NULL
wait/waitpid(pid,&st,opt)	read child; pid/0/-1	child exec path
void _exit(int status)	immediate exit, no stdio cleanup	close unused ends
int pipe(int p[2])	p[0] read, p[1] write	exec path fail
dup(olddf); dup2(oldd,new)	duplicate fd; dup2 returns new	exec path fail
fdopen(fd,mode); fileno(FILE*)	fd <-> FILE*	fclose 会 close fd
popen(cmd,type); pclose(fp)	shell command stream + wait	one-way
kill(pid,sig)	send signal; 0 or -1	不是只 kill
signal(sig,h)	install handler	handler 只设 flag
sigaction(sig,&sa,NULL)	metadata; 0/-1	mmap 常用
stat/fstat(...,&st)	addr or MAP_FAILED	不是 NULL; bytes not string
mmap(NULL,len,prot,flags,fd,off)	0/-1	unmap 后 ptr invalid
munmap(addr,len)	sharing / permissions	要匹配 fd mode
MAP_SHARED/PRIVATE; PROT_READ/WRITE		

Short API: pthread/sem/barrier	return / behavior	trap
form	compile + link pthread	两阶段都要
gcc ... -pthread	0 or error number	both lifetime
pthread_create(&tid,attr,fn,argv)	0/error; reclaim thread	join once
pthread_join(tid,&ret)	end calling thread	ret lifetime
pthread_exit(ret)	0/error	destroy 时 unlocked
pthread_mutex_init/destroy	0/error	保护 invariant
pthread_mutex_lock/unlock	0 or EBUSY	nonblocking
pthread_mutex_trylock	0/error	destroy 时 no waiters
pthread_cond_init/destroy	unlock+sleep+relock	while predicate
pthread_cond_wait(&cv,&m)	wake one/all	先改 state
signal/broadcast(&cv)	0/-1	pshared/-value
sem_init/destroy	wait decrements; post increments	trywait
sem_wait/trywait/post	inspect value	并发下只是瞬时值
sem_getvalue(&sv,&w)	fixed count sync	count >0; 错会挂
barrier_init/destroy/wait	parallel speedup	bottleneck
Amdahl: S=1/((1-P)/P+N)		

posix_spawn file actions mini
posix_spawn_file_actions_t fa;
posix_spawn_file_actions_init(&fa);
posix_spawn_file_actions_addopen2(&fa, oldfd, newfd); // 重定向 fd
posix_spawn_file_actions_addclose(&fa, fd); // 子进程关闭这个 fd
int rc = posix_spawn(&pid, path, &fa, NULL, argv, environ);
posix_spawn_file_actions_destroy(&fa);
if (rc == 0) waitpid(pid, &st, 0); // posix_spawn 直接返回错误码

input: scanf + fgets + getchar EOF
if (scanf("%d", &x) != 1) goto bad_input;
int ch;
while (ch = getchar()) != '\n' && ch != EOF {} // 清掉上一轮输入残留的换行
char line[N];
if (!fgets(line, sizeof line, stdin)) goto eof_or_error;
line[strlen(line, "\n")] = 0; // 去掉 fgets 保留下来的换行
for (int c; c = getchar(); c) { putchar(c); // 逐字节读取必须用 int, 才能区分 EOF

strtol / sscanf parsing
errno = 0;
char *end = NULL;
long v = strtol(argv[i], &end, 10);
if (end == argv[i]) goto no_digits;
if (*end != '\0') goto trailing_junk;
if (errno == ERANGE) goto out_of_range;

malloc + safe realloc
size_t cap = 16, len = 0;
T *a = malloc(cap * sizeof *a);
if (!a) goto fail;
if (len == cap) {
size_t nc = cap * 2;
T *tmp = realloc(a, nc * sizeof *a);
if (!tmp) goto fail; // realloc 失败时 a 仍然有效, 不能丢
a = tmp; cap = nc;

linked list insert
// 头插
new->next = head; head = new;
// 尾插
new->next = NULL;
if (!head) head = new;
else { cur = head; while (cur->next) cur = cur->next; cur->next = new
}

fork + execvp + waitpid
pid_t pid = fork();
if (pid < 0) goto fail;
if (pid == 0) {
execvp(argv[0], argv); // exec 成功不会返回
perorr(argv[0]);
_exit(127); // 子进程 exec 失败路径, 用 _exit 而不是 exit
}
int st;
waitpid(pid, &st, 0);
if (WIFEXITED(st)) code = WEXITSTATUS(st);

linked list delete via pointer-to-link
struct node *link = &head;
while (*link && (*link)-> != key) link = &(*link)->next;
if (*link) {
struct node *old = *link;
*link = old->next; // 先绕过 old, 再 free
free(old);

sigaction + main-loop cleanup
static volatile sig_atomic_t got_signal = 0;
static void handler(int sig) {
got_signal = sig; // handler 只记录事件/设 flag
}
struct sigaction sa = {0};
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0; // 可选: SA_RESTART / SA_NOCLDSTOP
if (sigaction(SIGTERM, &sa, NULL) < 0) goto fail;
if (sigaction(SIGINT, &sa, NULL) < 0) goto fail;
while (!got_signal) {
// normal work, or pause()/select()/read()
if (got_signal == SIGTERM got_signal == SIGINT) {
// main context 中统一 cleanup: close/free/kill/waitpid
}

free list / pointer graph
while (head) {
struct node *next = head->next;
free(head);
head = next;
for (int i=0; i<n; i++) {
T *curr = arr[i]; // arr[i] 是 T* 指针
if (!curr) continue;
T *front = curr->next; // 这是链表边, 不是数组里的 arr[i+1]
use(curr->field);

file stream + fscanf
FILE *fp = fopen(path, "r");
if (!fp) goto fail;
int id, rc; char name[32];
while ((rc = fscanf(fp, "%d %31s", &id, name)) == 2) {
operate_on(id, name);
if (rc != EOF) goto bad_record; // 读到半条记录或非法字段
if (ferror(fp)) goto ioerr;
fclose(fp);

fread binary records
FILE *fp = fopen(path, "rb");
if (!fp) goto fail;
struct rec rec;
while (fread(&rec, sizeof rec, 1, fp) == 1) {
operate_on(&rec);
}
if (ferror(fp)) goto ioerr; // 最后一条短读可能只是 EOF, 不一定是错误
fclose(fp);

strtok / frequency / lookup
char line[128];
if (fgets(line, sizeof line, fp)) {
line[strlen(line, "\n")] = 0;
for (char *tok = strtok(line, ","); tok; tok = strtok(NULL, ","))
use(tok);
}
int freq[256] = {0};
char c; while (c = getchar()) != EOF; { freq[(unsigned char)c]++;
const char *names[] = {"red", "green"}; int vals[] = {1, 2};
for (size_t i=0; i<n; i++) { strcmp(key, names[i])=0 return vals[i]
};

qsort comparator + signal flag
int cmp(const void *pa, const void *pb) {
const T *a = pa, *b = pb;
return (a->k > b->k) - (a->k < b->k);
}
static volatile sig_atomic_t stop;
static void h(int s) { (void)s; stop = 1; } // 信号处理函数里只设置标志位
struct sigaction sa = {0};
sa.sa_handler = h;
sigaction(SIGINT, &sa, NULL);

robust read/write partial + EINTR

pipe + dup2: child stdout -> parent
int pfd[2]; pipe(pfd);
if (fork()) {
dup2(pfd[1], \$STDOUT_FILENO);
close(pfd[0]); close(pfd[1]);
execvp(cmd[0], cmd);
_exit(127);
}
close(pfd[1]); // 父进程从 pfd[0] 读取
// 一直读到 EOF, 然后关闭 pfd[0], 最后等待子进程

two-way anonymous pipe model
int p2c[2], c2p[2];
pipe(p2c); pipe(c2p); // p[0]=read, p[1]=write
pid_t pid = fork();
if (pid == 0) {
close(p2c[1]); close(c2p[0]);
// child: read p2c[0], write c2p[1]
read(p2c[0], buf, sizeof buf);
write(c2p[1], reply, n);
close(p2c[0]); close(c2p[1]);
waitpid(pid, NULL, 0);

linear process chain + routing proof
for (int i=0; i<n-1; i++) {
pid_t pid = fork();
if (pid < 0) goto fail;
if (pid == 0) { my_id = i + 1; continue; } // 子进程可以继续创建下一环
child_pid = pid; break;
}
struct msg {int from:int to:int len;char msg[50];};
if (m.to == my_id) (deliver(m));
else if (m.to < my_id) {write(parent_write, &m, sizeof m);}
else {write(child_write, &m, sizeof m);}
// distance=abs(my_id-dst) 每次 -1, 所以无环且会到达

cleanup / SIGTERM discipline
static volatile sig_atomic_t stop;
static void term(int s) { (void)s; stop = 1; }
// 主循环检查 stop, 再统一清理资源
close_owned_pipe_fds();
free(state);
if (child_pid > 0) {
kill(child_pid, SIGTERM);
waitpid(child_pid, NULL, 0);
}

strtok / frequency / lookup
FILE *fp = popen(cmd, "r");
if (!fp) goto fail;
while (fgets(buf, sizeof buf, fp)) use(buf);
int st = pclose(fp);
free(state);
if (child_pid > 0) {
kill(child_pid, SIGTERM);
waitpid(child_pid, NULL, 0);
}

select over many fds
fd_set r;
int maxfd = -1;
FD_ZERO(&r);
for (int i=0; i<n; i++) {
if (fd[i] >= 0) {
FD_SET(fd[i], &r);
if (fd[i] > maxfd) maxfd = fd[i];
}
int rc = select(maxfd+1, &r, NULL, NULL, NULL);
if (rc < 0) { if (errno == EINTR) ; else goto fail; }
for (int i=0; i<n; i++) {
if (fd[i] >= 0 && FD_ISSET(fd[i], &r)) {
ssize_t nr = read(fd[i], buf, sizeof buf);
if (nr == 0) { close(fd[i]); fd[i] = -1; } // EOF
else if (nr > 0) handle(i, buf, nr);
}

pthread create args + join

```
struct arg args[N];
pthread_t t[N];

for (int i=0; i<N; i++) {
    args[i].id = i; // stable address; not &i
    pthread_create(&t[i], NULL, work, &args[i]);
}

for (int i=0; i<N; i++) {
    pthread_join(t[i], NULL); // join once
}
```

SIGCHLD reap zombie children

```
static volatile sig_atomic_t children_left = N;

static void chld(int sig) {
    (void)sig;
    int st;
    pid_t pid;
    while ((pid = waitpid(-1, &st, WNOHANG)) > 0) {
        children_left--;
        // WIFEXITED(st)/WEXITSTATUS(st)
        // WIFSIGNALED(st)/WTERMSIG(st)
    }
}

struct sigaction sa = {0};
sa.sa_handler = chld;
sa.sa_flags = SA_NOCLDSTOP;
sigemptyset(&sa.sa_mask);
sigaction(SIGCHLD, &sa, NULL);
```

function pointer / callback

```
int add(int a, int b) { return a + b; }

int (*fp)(int, int) = add;
int r = fp(2, 3);

typedef int (*op_t)(int, int);

int apply(int x, int y, op_t op) {
    return op(x, y);
}
// rule: int *(*f)(int) returns int*
// int (*)(int) is function pointer
```

pthread range split + shared output

```
typedef struct {
    const T *input; // 只读输入数据
    int lo, hi; // 本线程处理 [lo, hi)
    int *out; // 共享输出数组: 存匹配下标/结果编号
    int max_out; // 输出数组容量
    int *nout; // 当前已写入数量, 共享变量
    pthread_mutex_t *m; // 保护 out[] 和 *nout
} Arg;

int predicate(const T *input, int i) {
    // return 1 if input[i] satisfies condition
    return 0;
}

void *worker(void *p) {
    Arg *a = p;

    for (int i = a->lo; i < a->hi; i++) {
        if (predicate(a->input, i)) {
            pthread_mutex_lock(a->m);
            if (*a->nout < a->max_out) {
                a->out[(*a->nout)++] = i; // 输出顺序不保证
            }
            pthread_mutex_unlock(a->m);
        }
    }

    return NULL;
}

int run_parallel(const T *input, int total, int *out, int max_out) {
    pthread_t th[NTHREADS];
    Arg args[NTHREADS];

    int nout = 0;
    pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

    for (int t = 0; t < NTHREADS; t++) {
        args[t].input = input;
        args[t].lo = t * total / NTHREADS;
        args[t].hi = (t + 1) * total / NTHREADS;
        args[t].out = out;
        args[t].max_out = max_out;
        args[t].nout = &nout;
        args[t].m = &m;

        pthread_create(&th[t], NULL, worker, &args[t]);
    }

    for (int t = 0; t < NTHREADS; t++) {
        pthread_join(th[t], NULL);
    }

    pthread_mutex_destroy(&m);
    return nout;
}
```

queue enqueue/dequeue head-tail

```
typedef struct queue {
    Node *head;
    Node *tail;
```

```
} Queue;

int enqueue(Queue *q, int v) {
    if (!q) return 0;
    Node *n = new_node(v);
    if (!n) return 0;
    if (!q->tail) q->head = q->tail = n;
    else { q->tail->next = n; q->tail = n; }
    return 1;
}

int dequeue(Queue *q, int *out) {
    if (!q || !q->head || !out) return 0;
    Node *old = q->head;
    *out = old->value;
    q->head = old->next;
    if (!q->head) q->tail = NULL;
    free(old);
    return 1;
}
```

struct/union offset method

```
off = 0; maxalign = 1;
for each field:
    off = align_up(off, align(field));
    field_offset = off;
    off += sizeof(field);
sizeof_struct = align_up(off, maxalign);

// union: all members offset 0;
// size = max member size rounded up
// array: a[i].field = i*sizeof(T)+field_offset
```

lock ordering / deadlock avoidance

```
// acquire locks in one global order everywhere
pthread_mutex_lock(&a);
pthread_mutex_lock(&b);

// critical section using A and B

pthread_mutex_unlock(&b);
pthread_mutex_unlock(&a);

// no A->B in one thread and B->A in another
// no early return while holding mutex
// cond_wait releases only its own mutex
```

pthread condvar producer-consumer

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;
pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
int full = 0;

// producer
pthread_mutex_lock(&m);
while (full) pthread_cond_wait(&not_full, &m);
full = 1; // put item/update shared state
pthread_cond_signal(&not_empty);
pthread_mutex_unlock(&m);

// consumer
pthread_mutex_lock(&m);
while (!full) pthread_cond_wait(&not_empty, &m);
full = 0; // take item/update shared state
pthread_cond_signal(&not_full);
pthread_mutex_unlock(&m);
```

pthread queue + per-job completion

```
typedef struct {
    int complete, success;
    pthread_mutex_t m;
    pthread_cond_t cv;
} job_t;

job_t *q[N];
int head=0, tail=0, count=0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;

// submitter: enqueue job, then wait for this job
pthread_mutex_lock(&m);
q[tail] = job;
tail = (tail + 1) % N;
count++;
pthread_cond_signal(&not_empty);
pthread_mutex_unlock(&m);

pthread_mutex_lock(&job->m);
while (!job->complete) pthread_cond_wait(&job->cv, &job->m);
ok = job->success;
pthread_mutex_unlock(&job->m);

// worker: wait for queue, dequeue, process, notify owner
pthread_mutex_lock(&m);
while (count == 0) pthread_cond_wait(&not_empty, &m);
job_t *job = q[head];
head = (head + 1) % N;
count--;
pthread_mutex_unlock(&m);

ok = process(job);

pthread_mutex_lock(&job->m);
job->success = ok;
job->complete = 1;
pthread_cond_signal(&job->cv);
pthread_mutex_unlock(&job->m);
```

semaphore / barrier / Amdahl

```
sem_init(&s, 0, n); // n=资源数; 0=线程间共享
sem_wait(&s); // value=0: value-继续; value==0: 阻塞
// 使用资源: 最多允许 n 个线程同时进入
sem_post(&s); // value++; 释放资源/发出事件; 可能唤醒等待线程

// binary semaphore as mutex
sem_init(&s, 0, 1);
sem_wait(&s);
/* critical section: access shared state */
sem_post(&s);
// sem has no owner; mutex has owner

// event/counting use
sem_init(&done, 0, 0);
worker: sem_post(&done); // signal one completed event
main: sem_wait(&done); // wait for one event
// wait for k events: repeat sem_wait k times

// barrier: all threads finish phase1 before any enter phase2
pthread_barrier_init(&b, NULL, NTHREADS);
phase1();
pthread_barrier_wait(&b);
phase2();

// Amdahl's law: P=parallel fraction, 1-P=serial bottleneck
S(N) = 1 / ((1-P) + P/N);
max speedup as N->inf = 1/(1-P);
```

multi-cli/ser + select broadcast

```
typedef struct {
    pid_t pid;
    int rfd; // server reads: client -> server
    int wfd; // server writes: server -> client
    int alive;
} Client;

Client cs[N];
int alive_count = N;
char buf[BUFSZ];

while (alive_count > 0 && !stop) {
    fd_set rfd;
    FD_ZERO(&rfd);
    int maxfd = -1;

    // build read set: monitor all client->server pipes
    for (int i = 0; i < N; i++) {
        if (cs[i].alive) {
            FD_SET(cs[i].rfd, &rfd);
            if (cs[i].rfd > maxfd) maxfd = cs[i].rfd;
        }
    }

    int rc = select(maxfd + 1, &rfd, NULL, NULL, NULL);
    if (rc < 0) {
        if (errno == EINTR) continue;
        goto fail;
    }

    for (int i = 0; i < N; i++) {
        if (cs[i].alive && FD_ISSET(cs[i].rfd, &rfd)) {
            ssize_t nr = read(cs[i].rfd, buf, sizeof buf);

            if (nr == 0) { // EOF: client closed pipe
                close(cs[i].rfd);
                close(cs[i].wfd);
                cs[i].alive = 0;
                alive_count--;
            } else if (nr > 0) { // message from client i
                for (int j = 0; j < N; j++) {
                    if (j != i && cs[j].alive) {
                        write(cs[j].wfd, buf, nr); // broadcast to others
                    }
                }
            }
        }
    }

    // SIGTERM cleanup:
    // handler only sets stop=1; main closes fds, kill(pid, SIGTERM),
    // waitpid()
}
```

BST insert/search/free

```
typedef struct node {
    int v;
    struct node *left, *right;
} Node;

Node *new_node(int v) {
    Node *n = malloc(sizeof *n);
    n->v = v; n->left = n->right = NULL;
    return n;
}

void bst_insert(Node **root, int v) {
    while (*root) {
        if (v < (*root)->v) root = &(*root)->left;
        else root = &(*root)->right;
    }
    *root = new_node(v);
}

int bst_exists(Node *root, int x) {
    while (root) {
        if (x == root->v) return 1;
        root = (x < root->v) ? root->left : root->right;
    }
    return 0;
}

void bst_free(Node *root) {
    if (!root) return;
    bst_free(root->left);
    bst_free(root->right);
```

```
bst_free(root->right);
free(root);
}
```